Unit 3: Divide and Conquer

Unit 3 – Outline

Divide and Conquer

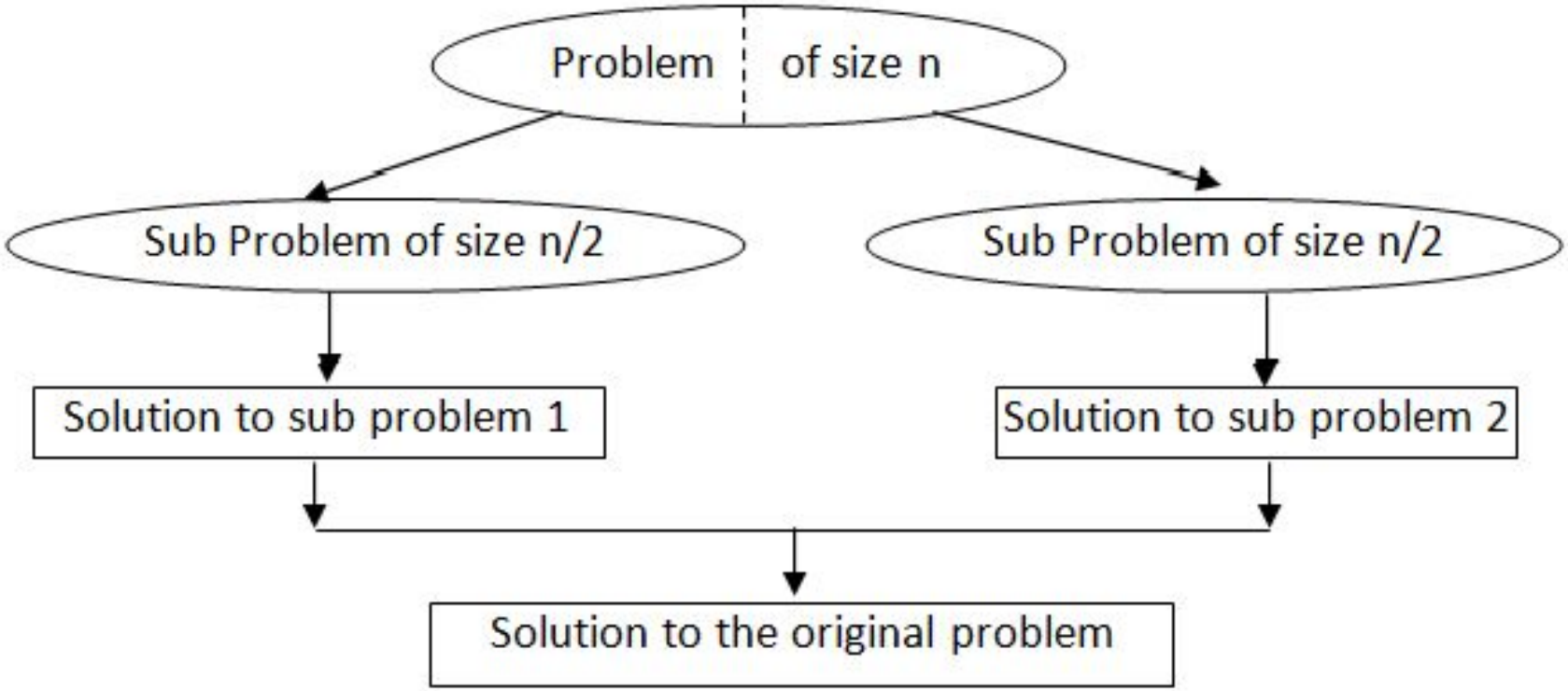
1. General method
2. Recurrence equation
3. Algorithm: Merge sort
4. Algorithm: Quick sort
5. Algorithm: Strassen's matrix multiplication
6. Multiplication of large integers

Unit 3 – Outline

Divide and Conquer

1. **General method**
2. Recurrence equation
3. Algorithm: Merge sort
4. Algorithm: Quick sort
5. Algorithm: Strassen's matrix multiplication
6. Multiplication of large integers

Divide and Conquer



Control Abstraction for Divide &Conquer

```
Algorithm DAndC(P)
{
    if Small(P) then return S(P);
    else
    {
        divide P into smaller instances P_1, P_2, ..., P_k, k ≥ 1;
        Apply DAndC to each of these subproblems;
        return Combine(DAndC(P_1),DAndC(P_2),...,DAndC(P_k));
    }
}
```

Module 3 – Outline

Divide and Conquer

1. General method
2. **Recurrence equation**
3. Algorithm: Merge sort
4. Algorithm: Quick sort
5. Algorithm: Strassen's matrix multiplication
6. Multiplication of large integers

Recurrence equation for Divide and Conquer

If the size of problem 'p' is n and the sizes of the 'k' sub problems are n_1, n_2 ….n_k, respectively, then

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \cdots + T(n_k) \quad + \quad f(n) & \text{otherwise} \end{cases}$$

Where,

- T(n) is the time for divide and conquer method on any input of size n and
- g(n) is the time to compute answer directly for small inputs.
- The function f(n) is the time for dividing the problem 'p' and combining the solutions to sub problems.

Recurrence equation for Divide and Conquer

- Generally, an instance of size **n** can be divided into **b** instances of size **n/b**,
- Assuming $n = b^k$,

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

where f(n) is a function that accounts for the time spent on dividing the problem into smaller ones and on combining their solutions.

Solving recurrence relation using **Master theorem**

It states that, in recurrence equation $T(n) = aT(n/b) + f(n)$,

If $f(n) \in \Theta(n^d)$ where $d \geq 0$ then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log_b n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too. Example:

$$A(n) = 2A(n/2) + 1.$$

Here $a = 2$, $b = 2$, and $d = 0$; hence, since $a > b^d$,

$$A(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n).$$

Module 3 – Outline

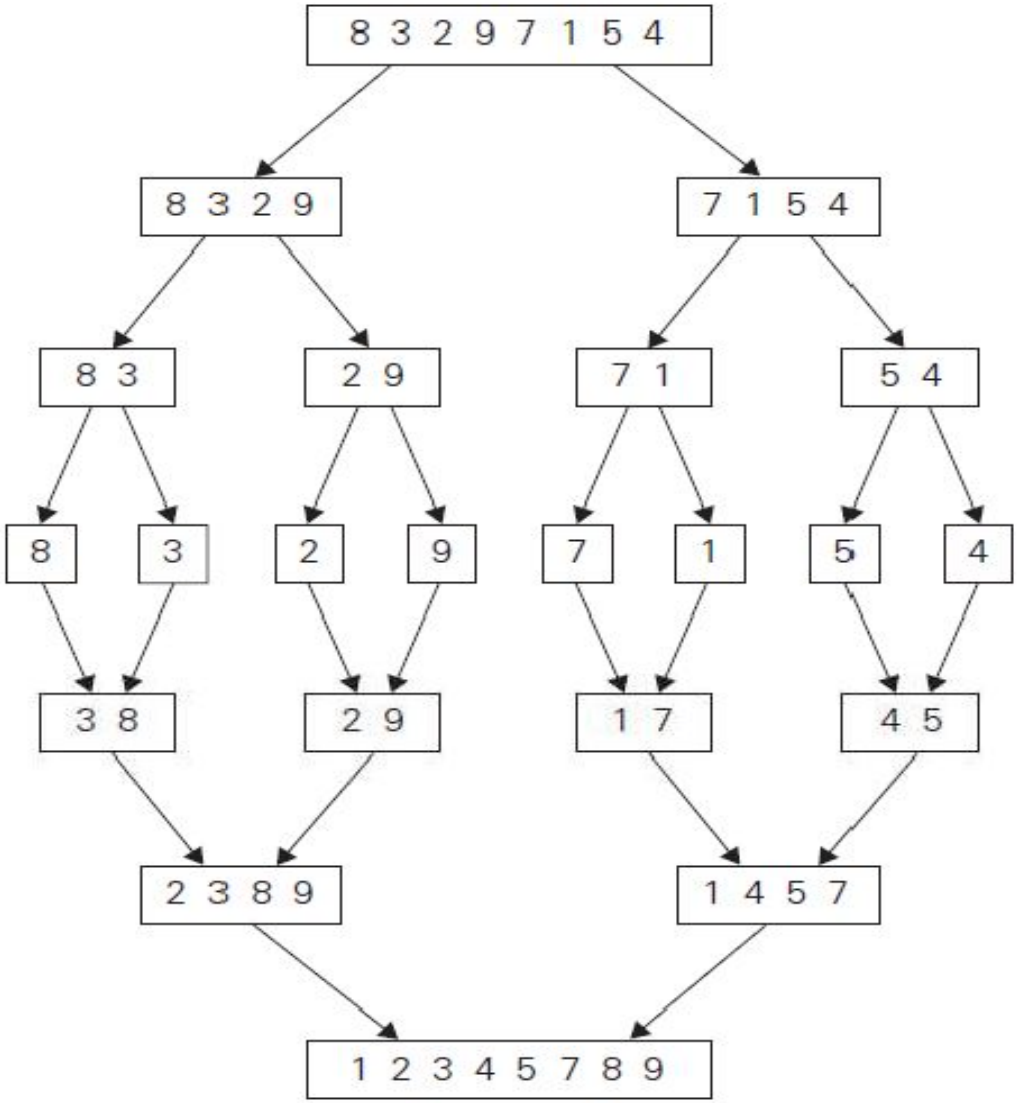
Divide and Conquer

1. General method
2. Recurrence equation
3. **Algorithm: Merge sort**
4. Algorithm: Quick sort
5. Algorithm: Strassen's matrix multiplication
6. Multiplication of large integers

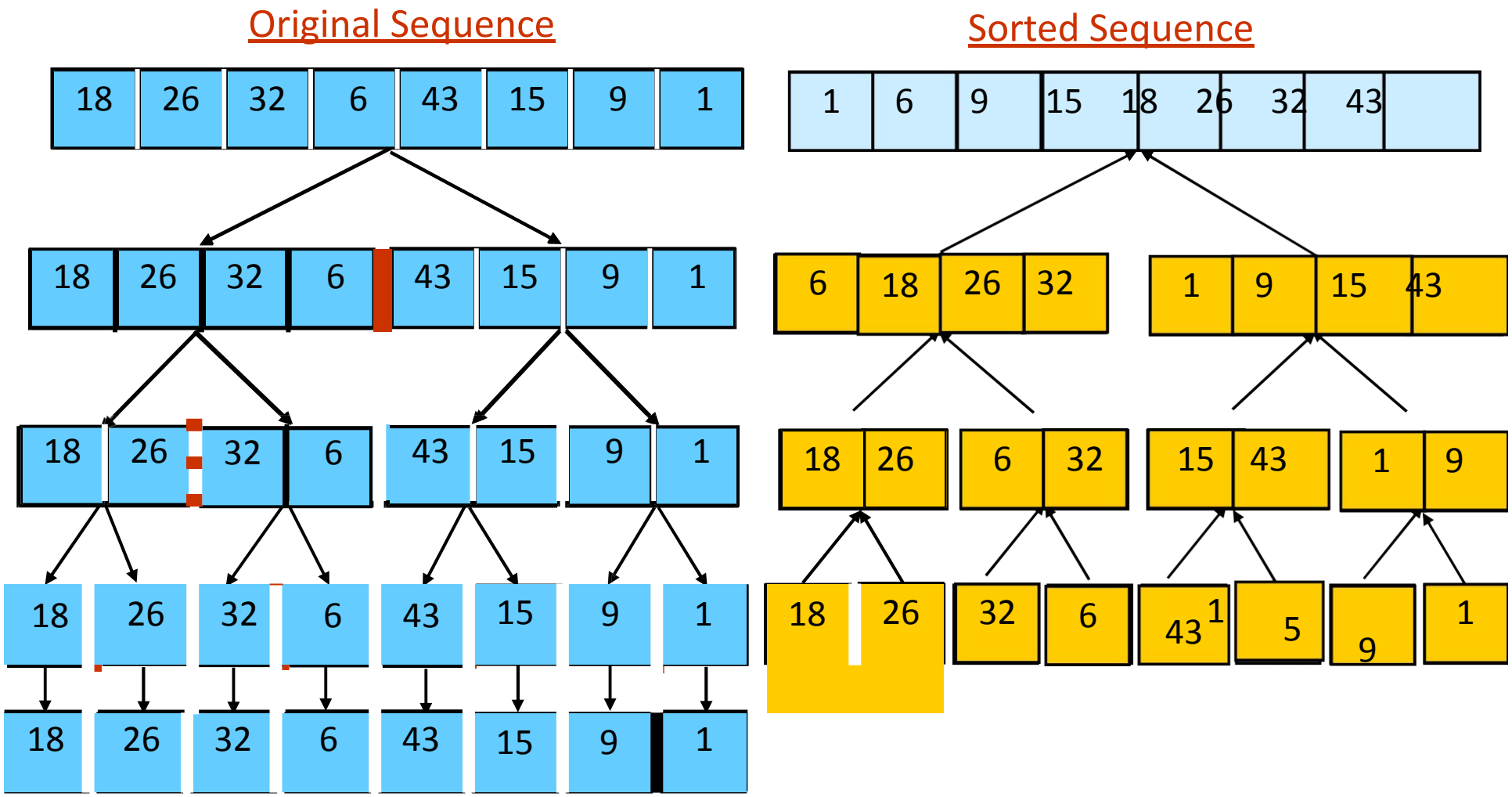
Merge Sort

- Merge sort is a perfect example of divide-and conquer technique.
- It sorts a given array by
  - dividing it into two halves,
  - sorting each of them recursively, and
  - then merging the two smaller sorted arrays into a single sorted one.

Merge sort - example



Merge Sort - Example



Merge Sort

```
Algorithm Mergesort(a, low, high)
{
    if(low<high)
    {
        //Divide Problem into subproblems
        //Find where to split the array
            mid=(low+high)/2
        //solve the subproblems
        Mergesort(a, low, mid)
        Mergesort(a, mid+1, high)
        //combine the solutions of subproblems
        Merge(a, low, mid, high)
    }
}
```

Merge

```
Algorithm Merge(a, low, mid, high)
{
      int b[];
      i=low, j=mid+1, k=low, l
     while(i<= mid and j<=high)
    {
    if(a[i]<=a[j])
    {
         b[k]=a[i]
         i++
          }
    else
    {
         b[k]=a[j];
         j++
    }
    k++
    }
    while(i<=mid)
    {
        b[k++]=a[i]
         i++
     }
      while(j<=high)
      {
          b[k++]=a[j]
          j++
       }
       for (l=low; l<high; l++)
      {
           a[l]=b[l]
       }
}
```

Analysis

- Basic operation - key comparison.
- Best Case, Worst Case, Average Case exists?
  - Execution does not depend on the order of the data
  - Best case and average case runtime are the same as worst case runtime.
- Worst case:
  - During key comparison, neither of the two arrays becomes empty before the other one contains just one element

Analysis – Worst Case

- Assuming for simplicity that total number of elements **n** is a power of 2, the recurrence relation for the number of key comparisons $C(n)$ is

$$C(n) = 2C(n/2) + C_{merge}(n) \quad \text{for } n > 1, \quad C(1) = 0.$$

- $C_{merge}(n)$ - the number of key comparisons performed during the merging stage.
- At each step, exactly one comparison is made, total comparisons are (n-1)

$$C_{worst}(n) = 2C_{worst}(n/2) + n - 1 \quad \text{for } n > 1, \quad C_{worst}(1) = 0.$$

Analysis

$$C_{worst}(n) = 2C_{worst}(n/2) + n - 1 \quad \text{for } n > 1, \quad C_{worst}(1) = 0.$$

- Here a = 2, b = 2, f (n) = n-1 = Θ (n) => d = 1.
- Therefore $2 = 2^1$, case 2 holds in the master theorem
- $C_{worst}(n) = \Theta(n^d \log n) = \Theta(n^1 \log n) = \Theta(n \log n)$
- Therefore $C_{worst}(n) = \Theta(n \log n)$

Master theorem

It states that, in recurrence equation T(n) = aT(n/b) + f(n),
If $f(n) \in \Theta(n^d)$ where $d \geq 0$ then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log_b n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Advantages

- Number of comparisons performed is nearly optimal.
- For large n, the number of comparisons made by this algorithm in the average case turns out to be about 0.25n less and hence is also in $\Theta(n \log n)$.
- Mergesort will never degrade to $O(n^2)$
- Another advantage of mergesort over quicksort is its stability.

  (A **sorting algorithm** is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input array to be sorted.)

Limitations

- The principal shortcoming of mergesort is the linear amount $O(n)$ of extra storage the algorithm requires.
- Though merging can be done in-place, the resulting algorithm is quite complicated and of theoretical interest only.

Module 3 – Outline

Divide and Conquer

1. General method
2. Recurrence equation
3. Algorithm: Merge sort
4. **Algorithm: Quick sort**
5. Algorithm: Strassen's matrix multiplication
6. Multiplication of large integers

Quick Sort

- It is a Divide and Conquer method
- Sorting happens in Divide stage itself.
- Charles Antony Richard Hoare (also known as Tony Hore), prominent British computer scientist invented quicksort.

Quick Sort

- Quicksort divides (or partitions) array according to the value of some pivot element A[s]
- Divide-and-Conquer:
  - If n=1 terminate (every one-element list is already sorted)
  - If n>1, partition elements into two; based on pivot element, such that

$$\underbrace{A[0]\ldots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1]\ldots A[n-1]}_{\text{all are } \geq A[s]}$$

Quick Sort

$$\underbrace{A[0]\ldots A[s-1]}_{\text{all are } \le A[s]} \quad A[s] \quad \underbrace{A[s+1]\ldots A[n-1]}_{\text{all are } \ge A[s]}$$

```
Algorithm Quicksort(a, p, q)
{
    if(p < q)
    {
        j=partition(a, p, q);      // partition array into parts
        quicksort(a, p, j-1);      // sort left part of array
        quicksort(a, j+1, q);      // sort right part of array
    }
}
```

How do we partition?

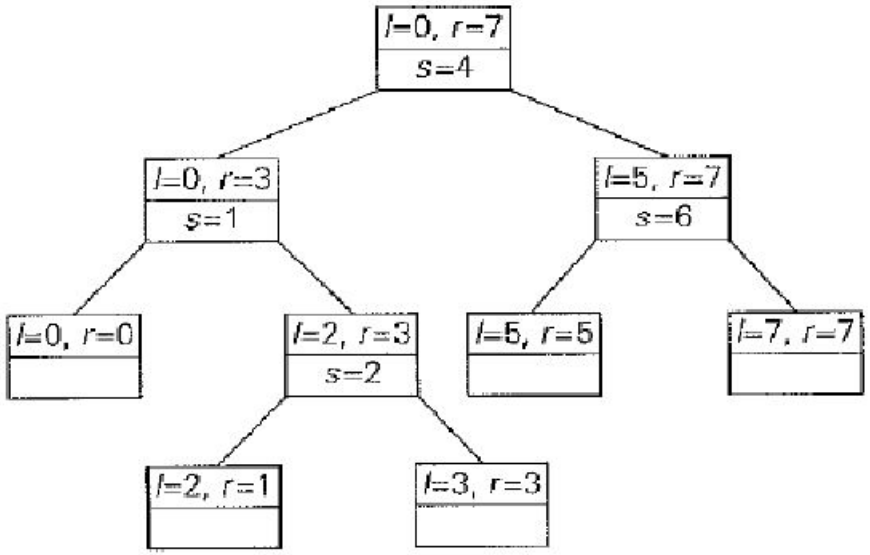
- There are several different strategies for selecting a pivot and partitioning.
- We use the sophisticated method suggested by C.A.R. Hoare, the inventor of quicksort.

- Select the subarray's first element as pivot: **p = A[0].**
- Now scan the subarray from both ends, comparing the subarray's elements to the pivot.

How do we partition?

```
Algorithm partition(a[], m,  p)
{
      v=a[m];       // first element is pivot element
      i=m+1;
      j=p;
      while(i < j)
      {
      while(a[i] < v and i<=p)
            i++;
      while(a[j] > v and j>=m+1)
            j--;
      if(i < j)
            swap(a[i], a[j]) //swap the contents
      }
      swap(a[m], a[j])     //swap pivot and a[j]
      return j;
}
```

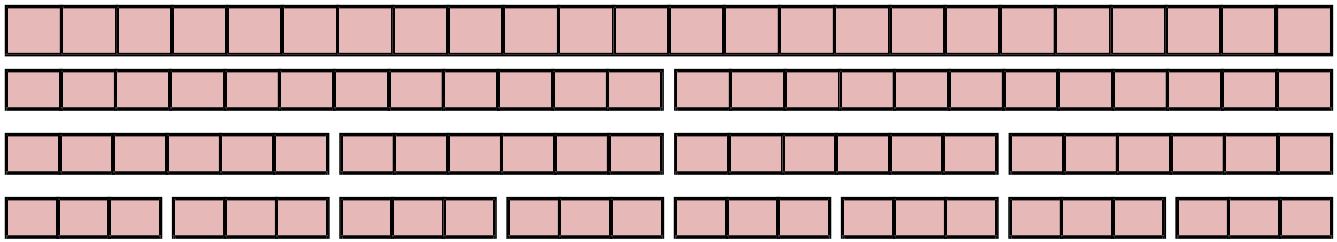
0	1	2	3	4	5	6	7
	i						*i*
5	3	1	9	8	2	4	7
			i			*i*	
5	3	**1**	9	8	2	4	7
			i			*i*	
5	3	**1**	4	8	2	9	7
				i	*i*		
5	3	**1**	4	8	2	9	7
				i	*i*		
5	3	**1**	4	2	8	9	7
				i	*i*		
5	3	**1**	4	2	8	9	7
2	3	**1**	**4**	**5**	8	9	7
	i		*i*				
2	3	**1**	4				
	i	*i*					
2	3	**1**	4				
	i	*i*					
2	1	3	4				
	i	*i*					
2	1	3	4				
1	**2**	3	4				
1							
			i j				
		3	4				
		i	*i*				
		3	4				
			4				
						i	*i*
					8	9	7
						i	*i*
					8	7	9
						i	*i*
					8	7	9
					7	**8**	9
					7		
							9



(b)

Analysis

- Basic Operation : Key Comparison
- Best case exists
  - all the splits happen in the middle of subarrays,
  - So the depth of the recursion is $\log_2 n$



$$C_{best}(n) = 2C_{best}(n/2) + n \quad \text{for } n > 1, \quad C_{best}(1) = 0.$$

  - As per Master Theorem, $C_{best}(n) \in \Theta(n \log_2 n)$;

Best case scenario for the Quick Sort algorithm

- The **Quick Sort** algorithm performs the ***best*** when the **pivot** is the middle value in the **array.**
- In the **best case**, the **input array** is **divided** into **2 arrays** each containing **approximately n/2** elements.

Summary of the **execution**:

```
QuickSort( input array with n elements )
{   ....
    partition( a (array with n elements) );
    QuickSort( array with n/2 elements );
    QuickSort( array with n/2 elements );
}
```

Therefore:

$$T(n) = partition(n) + T(n/2) + T(n/2)$$
$$= partition(n) + 2*T(n/2) \qquad // \ partition(n) = n$$
$$= n + 2*T(n/2)$$
$$= 2*T(n/2) + n$$
$$T(1) = 1 \qquad // \text{ For the if-check operation}$$

Best case for the Quick Sort algorithm

$T(n) = 2*T(n/2) + n$ // $T(n/2) = 2*T(n/4) + (n/2)$

$= 2*[\ 2*T(n/4) + n/2\ ] + n$
$= 2^2*T(n/4) + n + n$
$= 2^2*T(n/4) + 2n$ // $T(n/4) = 2*T(n/8) + (n/4)$
$= 2^2*[\ 2*T(n/8) + (n/4)\ ] + 2n$
$= 2^3*T(n/8) + 2^2*(n/4) + 2n$
$= 2^3*T(n/8) + n + 2n$
$= 2^3*T(n/8) + 3n$
$= 2^4*T(n/16) + 4n$
and so on....

$= 2^k*T(n/(2^k)) + k*n$ // Keep going until: $n/(2^k) = 1 \iff n = 2^k$
$= 2^k*T(1) + k*n$
$= 2^k*1 + k*n$
$= 2^k + k*n$ // $n = 2^k$
$= n + k*n$
$= n + (\lg(n))*n$
$= n*(\ \lg(n) + 1\ )$
$\sim= n*\lg(n))$
$T(n) = n*\log(n)$

Analysis

- Worst Case
  - Splits will be skewed to the extreme
  - This happens if the input is already sorted
- In the worst case, partitioning always divides the size n array into these three parts:
  - A length one part, containing the pivot itself
  - A length zero part, and
  - A length n-1 part, containing everything else
- Recurring on the length n-1 part requires (in the worst case) recurring to depth n-1

Analysis

Worst Case

- if A[0..n − 1] is a strictly increasing array and we use A[0] as the pivot,
  - the left-to-right scan will stop on A[1] while the right-to-left scan will go all the way to reach A[0], indicating the split at position 0
  - n + 1 comparisons required

Total comparisons

$$C_{worst}(n) = (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2).$$

worst case performance of **Quick Sort**

Running time of **partition(array of n elements)** is equal to:

= $(n - 1)$ (while loop) + **1** (copying the pivot back)
= n

$T(n) = partition(n) + T(n-1)$ // partition(n)
$= n = n + T(n-1)$
$= T(n-1) + n$

$T(1) = 1$ // For the if-check operation

Solving the **recurrence relation**:

$T(n) = T(n-1) + n$ // $T(n-1) = T(n-2) + (n-1)$
$= T(n-2) + (n-1) + n$
$= T(n-3) + (n-2) + (n-1) + n$
$= \ldots$
$= T(1) + 2 + 3 + \ldots + (n-1) + n$ // $T(1) = 1$
$= 1 + 2 + 3 + \ldots + (n-1) + n$
$= \frac{n(n+1)}{2}$

Therefore: $T(n) = \frac{n(n+1)}{2} = O(n^2)$

Analysis

Average Case

- Let $C_{avg}(n)$ be the average number of key comparisons made by quicksort on a randomly ordered array of size n.
- A partition can happen in any position s $(0 \leq s \leq n-1)$
- n+1 comparisons are required for partition.
- After the partition, the left and right subarrays will have s and $n - 1 - s$ elements, respectively.

Analysis

Average Case

- Assuming that the partition split can happen in each position s with the same probability 1/n, we get

$$C_{avg}(n) = \frac{1}{n}\sum_{s=0}^{n-1}[(n+1) + C_{avg}(s) + C_{avg}(n-1-s)] \quad \text{for } n > 1,$$

$$C_{avg}(0) = 0, \quad C_{avg}(1) = 0.$$

$$C_{avg}(n) \approx 2n \ln n \approx 1.39n \log_2 n.$$

Pros and Cons

- Pros
  - Good average case time complexity
- Cons
  - It is not stable.
  - It requires a stack to store parameters of subarrays that are yet to be sorted.

Module 3 – Outline

Divide and Conquer

1. General method
2. Recurrence equation
3. Algorithm: Merge sort
4. Algorithm: Quick sort
5. **Algorithm: Strassen's matrix multiplication**
6. Multiplication of large integers

Matrix Multiplication

Direct Method to multiply two 2 X 2 matrices:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} \quad \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$ $\qquad$ $C_{11} = 1 * 5 + 2 * 7 = 19$

$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$ $\qquad$ $C_{12} = 1 * 6 + 2 * 8 = 22$

$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$ $\qquad$ $C_{21} = 3 * 5 + 4 * 7 = 43$

$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$ $\qquad$ $C_{22} = 3 * 6 + 4 * 8 = 50$

Matrix Multiplication

Direct Method to multiply two 2 X 2 matrices example:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$

$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$

$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$

$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$

Let us Count the Number of Multiplications

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$C_{11} = 1 * 5 + 2 * 7 = 19$

$C_{12} = 1 * 6 + 2 * 8 = 22$

$C_{21} = 3 * 5 + 4 * 7 = 43$

$C_{22} = 3 * 6 + 4 * 8 = 50$

Let us Count the Number of Additions

Number of Multiplications = **8**

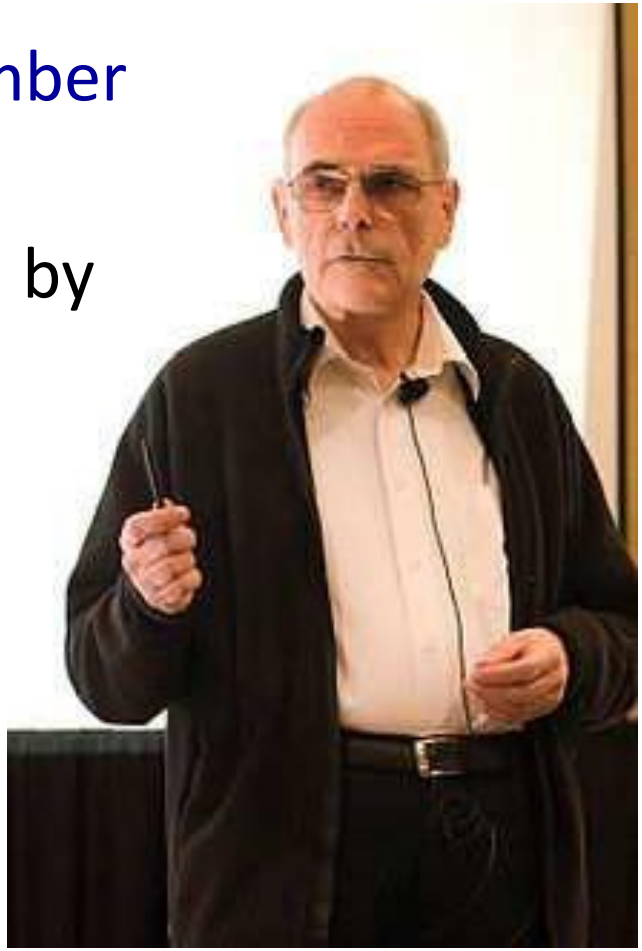
Number of Additions = **4**

In traditional method of multiplying two 2 * 2 Matrices, we require 8 Multiplications and 4 Additions

Stressen's matrix multiplication

Multiplication of 2×2 matrices:

- By using divide-and-conquer approach we can reduce the number of multiplications.
- Such an algorithm was published by V. Strassen in 1969.



Strassen's Method: Let us consider the Strassen's Method of multiplying two 2 * 2 matrices

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$D = A_{11} * (B_{12} - B_{22})$

$E = A_{22} * (B_{21} - B_{11})$

$F = (A_{21} + A_{22}) * B_{11}$

$G = (A_{11} + A_{12}) * B_{22}$

$H = (A_{21} - A_{11}) * (B_{11} + B_{12})$

$I = (A_{12} - A_{22}) * (B_{21} + B_{22})$

$J = (A_{11} + A_{22}) * (B_{11} + B_{22})$

$C_{11} = E + I + J - G$

$C_{12} = D + G$

$C_{21} = E + F$

$C_{22} = D + H + J - F$

$D = 1 * (6 - 8) = -2$

$E = 4 * (7 - 5) = 8$

$F = (3 + 4) * 5 = 35$

$G = (1 + 2) * 8 = 24$

$H = (3 - 1) * (5 + 6) = 22$

$I = (2 - 4) * (7 + 8) = -30$

$J = (1 + 4) * (5 + 8) = 65$

$C_{11} = 8 + (-30) + 65 - 24 = 19$

$C_{12} = -2 + 24 = 22$

$C_{21} = 8 + 35 = 43$

$C_{22} = -2 + 22 + 65 - 35 = 50$

Number of Multiplications = 7 and Number of Additions = 18

Strassen's Method

In the traditional method of multiplying two 2 * 2 matrices, we require 8 multiplications and 4 additions.

In the Strassen's method of multiplying two 2 * 2 matrices, we require 7 multiplications and 18 additions.

Note: Time taken to perform addition is significantly less compared to time taken to perform multiplication.

The time saved is very less when Strassen's Method is used only once to multiply two 2 * 2 matrices.

But the time saved will be more and significant when Strassen's Method is used to multiply large number of two 2 * 2 matrices.

Strassen's Matrix Multiplication

Consider the following 4 * 4 Matrix

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix}$$

This 4 * 4 Matrix can be divided into four 2 * 2 Matrices

$$\left[\begin{array}{cc|cc} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ \hline A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{array}\right]$$

Similarly, an 8 * 8 Matrix can be divided into four 4 * 4 matrices, which in turn can be further divided into four 2 * 2 matrices.

Similarly, a 16 * 16 Matrix can be divided into four 8 * 8 matrices, each 8 * 8 matrix can be divided into four 4 * 4 matrices, which in turn can be further divided into four 2 * 2 matrices.

Strassen's Matrix Multiplication (Contd..)

Consider the multiplication of two 4 * 4 Matrix using Divide and Conquer

$$\underbrace{\begin{bmatrix} 2*2 & 2*2 \\ 2*2 & 2*2 \end{bmatrix}}_{A\,[4*4]} * \underbrace{\begin{bmatrix} 2*2 & 2*2 \\ 2*2 & 2*2 \end{bmatrix}}_{B\,[4*4]} = \underbrace{\begin{bmatrix} 2*2 & 2*2 \\ 2*2 & 2*2 \end{bmatrix}}_{C\,[4*4]}$$

Let us Count the Number of 2 * 2 Matrix Multiplications

$$\begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} + \begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} = \begin{bmatrix} 2*2 \end{bmatrix}$$

$$\begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} + \begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} = \begin{bmatrix} 2*2 \end{bmatrix}$$

$$\begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} + \begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} = \begin{bmatrix} 2*2 \end{bmatrix}$$

$$\begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} + \begin{bmatrix} 2*2 \end{bmatrix} * \begin{bmatrix} 2*2 \end{bmatrix} = \begin{bmatrix} 2*2 \end{bmatrix}$$

Number of 2 * 2 Matrix Multiplications = 8

If we use Strassen's Method for these 8 multiplications, the time saved increases.

Multiplying two 8 * 8 Matrices using Divide and Conquer needs 64 2 * 2 Matrix multiplications. If we use Strassen's Method for these 64 multiplications, the time saved further increases.

As the dimensions of the Matrices increases, the time saved by Strassen's Method also increases exponentially.

Strassen's Matrix Multiplication (Contd..)

If Strassen's Method is used to multiply 2 * 2 matrices only once, it saves a very less time and is almost insignificant.

If Strassen's Method is used to multiply large number of 2 * 2 matrices, then the time saved is huge.

During the multiplication of two big dimension matrices, we apply Divide and Conquer to get large number of 2 * 2 matrices. Then, we go on multiplying these 2 * 2 matrices using Strassen's method and hence enormous amount of time is saved.

In Summary,
Strassen's Method is almost insignificant without Divide and Conquer.
Strassen's Method becomes very much significant with Divide and Conquer.

Analysis of Traditional Method

In the Traditional Method, we have 8 multiplications and multiplication is the primary operation and also divide and conquer is used to multiply large-dimension matrices. Hence, the recurrence relation is as shown below:

$$T(N) = \begin{cases} 1 & \text{if } N = 1 \\ 8 * T\left[\frac{N}{2}\right] & \text{otherwise} \end{cases}$$

$$T(N) = 8 * T\left[\frac{N}{2}\right]$$

Assume $N = 2^k$

$$T(2^k) = 8 * T\left[\frac{2^k}{2}\right]$$

We know that $\frac{2^k}{2} = 2^{k-1}$

$$T(2^k) = 8 * T(2^{k-1}) \quad \cdots\cdots (1)$$

Substituting k by k-1 in (1), we get

$$T(2^{k-1}) = 8 * T(2^{k-2}) \quad \cdots\cdots (2)$$

Substituting (2) in (1), we get

$$T(2^k) = 8 * \left[8 * T(2^{k-2})\right]$$

$$T(2^k) = 8^2 * T(2^{k-2})$$

$$T(2^k) = 8^3 * T(2^{k-3})$$

$$T(2^k) = 8^4 * T(2^{k-4})$$

$$\vdots$$

$$T(2^k) = 8^k * T(2^{k-k})$$

$$T(2^k) = 8^k * T(2^0)$$

$$T(2^k) = 8^k * T(1)$$

$$T(2^k) = 8^k * 1$$

$$T(2^k) = 8^k$$

We know $N = 2^k$ and $k = \log_2 N$

$$T(N) = 8^{\log_2 N}$$

$$T(N) = N^{\log_2 8} \qquad a^{\log_b c} = c^{\log_b a}$$

We know that $\log_2 8 = 3$

$$T(N) = N^3$$

$$\boxed{T(N) = \theta(N^3)}$$

Analysis of Strassen's Method

In Strassen's Method, we have 7 multiplications and multiplication is the primary operation and also divide and conquer is used to multiply large-dimension matrices. Hence, the recurrence relation is as shown below:

$$T(N) = \begin{cases} 1 & \text{if } N = 1 \\ 7 \cdot T\left[\frac{N}{2}\right] & \text{otherwise} \end{cases}$$

$$T(N) = 7 \cdot T\left[\frac{N}{2}\right]$$

Assume $N = 2^k$

$$T(2^k) = 7 \cdot T\left[\frac{2^k}{2}\right]$$

We know that $\frac{2^k}{2} = 2^{k-1}$

$$T(2^k) = 7 \cdot T(2^{k-1}) \quad \cdots\cdots (1)$$

Substituting k by k-1 in (1), we get

$$T(2^{k-1}) = 7 \cdot T(2^{k-2}) \quad \cdots\cdots (2)$$

Substituting (2) in (1), we get

$$T(2^k) = 7 \cdot \left[8 \cdot T(2^{k-2})\right]$$

$$T(2^k) = 7^2 \cdot T(2^{k-2})$$

$$T(2^k) = 7^3 \cdot T(2^{k-3})$$

$$T(2^k) = 7^4 \cdot T(2^{k-4})$$

$$\vdots$$

$$T(2^k) = 7^k \cdot T(2^{k-k})$$

$$T(2^k) = 7^k \cdot T(2^0)$$

$$T(2^k) = 7^k \cdot T(1)$$

$$T(2^k) = 7^k \cdot 1$$

$$T(2^k) = 7^k$$

We know $N = 2^k$ and $k = \log_2 N$

$$T(N) = 7^{\log_2 N}$$

$$T(N) = N^{\log_2 7} \qquad a^{\log_b c} = c^{\log_b a}$$

We know that $\log_2 7 = 2.807$

$$T(N) = N^{2.807}$$

$$\boxed{T(N) = \theta(N^{2.807})}$$

Analysis of Strassen's Method

Suppose if we consider both **multiplication and addition.** The resulting recurrence ration T(n) is

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + an^2 & n > 2 \end{cases}$$

Note: No. of addition/ subtraction Operations $18(n/2)^2 = an^2$

where a and b are constants. Working with this formula, we get

$$\begin{aligned} T(n) &= an^2[1 + 7/4 + (7/4)^2 + \cdots + (7/4)^{k-1}] + 7^k T(1) \\ &\leq cn^2(7/4)^{\log_2 n} + 7^{\log_2 n}, \ c \text{ a constant} \\ &= cn^{\log_2 4 + \log_2 7 - \log_2 4} + n^{\log_2 7} \\ &= O(n^{\log_2 7}) \approx O(n^{2.81}) \end{aligned}$$

Unit 2 – Outline

Divide and Conquer

1. General method
2. Recurrence equation
3. Algorithm: Merge sort
4. Algorithm: Quick sort
5. Algorithm: Strassen's matrix multiplication
6. Multiplication of large integers

Multiplication of large integers

Multiplying large integers

$$\begin{array}{r} 0981 \\ \times \quad \underline{1234} \\ 1210554 \end{array}$$

The traditional method algorithm takes $O(n^2)$

Divide and Conquer method:

$w = 09 \qquad y = 12$

$x = 81 \qquad z = 34$

$981 = 10^2 w + x$

$1234 = 10^2 y + z$

Multiplication of large integers (Contd..)

The required product can be computed as

$$
\begin{aligned}
981 \text{ X } 1234 &= (10^2 w + x) * (10^2 y + z) \\
&= 10^4 wy + 10^2 wz + 10^2 xy + xz \\
&= 10^4 wy + 10^2 (wz + xy) + xz \quad \text{---- (1)} \\
&= 1080000 + 127800 + 2754 \\
&= 1210554
\end{aligned}
$$

The above procedure still needs four half-size multiplications (for wy, wz, xy, xz).

Is it possible to obtain wz + xy at the cost of a single multiplication?

$$
\begin{aligned}
r &= (w + x) * (y + z) \\
&= wy + (wz + xy) + xz
\end{aligned}
$$

Multiplication of large integers (Contd..)

After only one multiplication, we obtain the sum of all three terms needed to calculate the designed product.

$P = wy = 09 * 12 = 108$

$Q = xz = 81 * 34 = 2754$

$R = (w + x) * (y + z)$

$w = 09, \; y = 12$

$x = 81, \; z = 34$

Finally, $981 * 1234 = 10^4 wy + 10^2 (wz + xy) + xz$ ---- (1)

$= 10^4 p + 10^2 (r - p - q) + q$

$= 1080000 + 127800 + 2754$

$= 1210554$

Thus, the product of 981 and 1234 can be reduced to three multiplications of two-digit numbers.

09 * 12
81 * 34 Total three multiplications only
90 * 46

In General: $wy10^{2(n/2)}+(wz+xy)10^{n/2}+xz$

When the operands are large, the time required for the additions and shift becomes negligible compared to the time taken by a single multiplication.

Multiplication of large integers (Contd..)

In general any integer of 'n' digits can be represented as:
where a and b are n digit integers, divided into 2 halves, (a=w & x, b=y & z)

$a = w*10^{n/2} + x$
$b = y*10^{n/2} + z$

$a = 981$

$b = 1234$

$a = wx$ ($w = 09$ & $x = 81$)

$b = yz$ ($y = 12$ & $z = 34$)

$$a * b = (w*10^{n/2} + x)*(y*10^{n/2} + z)$$
$$= (w*10^{n/2})(y*10^{n/2}) + wz*10^{n/2} + xy*10^{n/2} + xz$$
$$a * b = wy*10^{2(n/2)} + (wz+xy)*10^{n/2} + xz$$

In General: $a * b = wy*10^{2(n/2)} + (wz+xy)*10^{n/2} + xz$

Analysis of Multiplication of large integers

- How many digit multiplications does this algorithm make?
- Since multiplication of n-digit numbers requires three multiplications of n/2-digit numbers, the recurrence for the number of multiplications M(n) is

$$\mathbf{M(n) = 3M(n/2) \quad \text{for } n > 1, \quad M(1) = 1}$$

Solving it by backward substitutions for $n = 2^k$ yields

$$M(2^k) = 3M(2^{k-1}) = 3[3M(2^{k-2})] = 3^2 M(2^{k-2})$$
$$= \cdots = 3^i M(2^{k-i}) = \cdots = 3^k M(2^{k-k}) = 3^k.$$

Since $k = \log_2 n$,

$$M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}.$$

(On the last step, we took advantage of the following property of logarithms: $a^{\log_b c} = c^{\log_b a}$.)

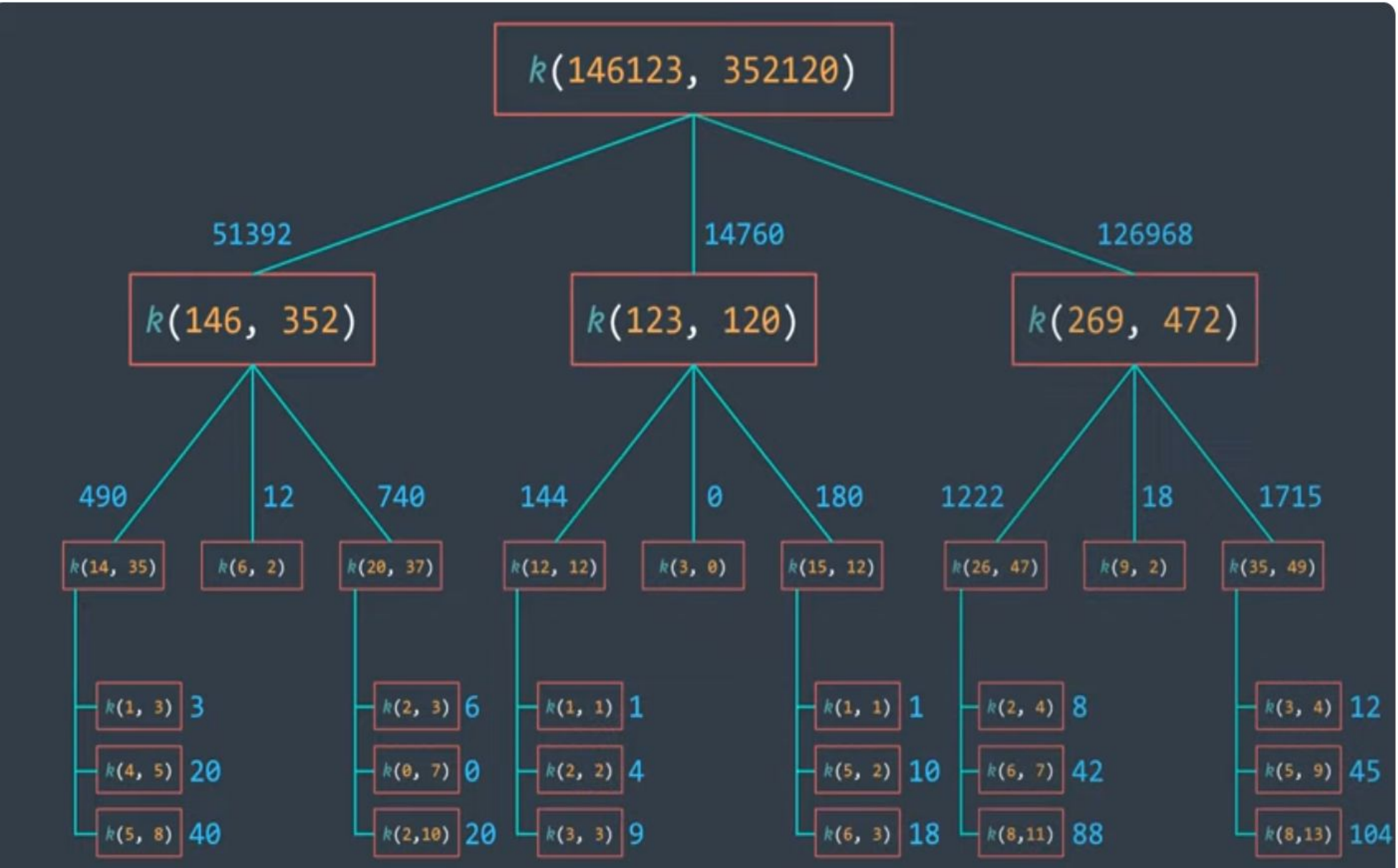
Analysis of Multiplication of large integers

- But what about additions and subtractions?
- Let A(n) be the number of digit additions and subtractions executed by the above algorithm in multiplying two n-digit decimal integers.
- Besides 3A(n/2) of these operations needed to compute the three products of n/2-digit numbers, the above formulas require five additions and one subtraction.
- Hence, we have the recurrence

$$A(n) = 3A(n/2) + cn \quad \text{for } n > 1, \; A(1) = 1$$

Applying the Master Theorem,
we obtain $A(n) \in (n\log_2 3)$

which means that the total number of additions and subtractions have the same asymptotic order of growth as the number of multiplications.

Example: 146123 * 352120

$wy10^{2(n/2)}+(wz+xy)10^{n/2}+xz$